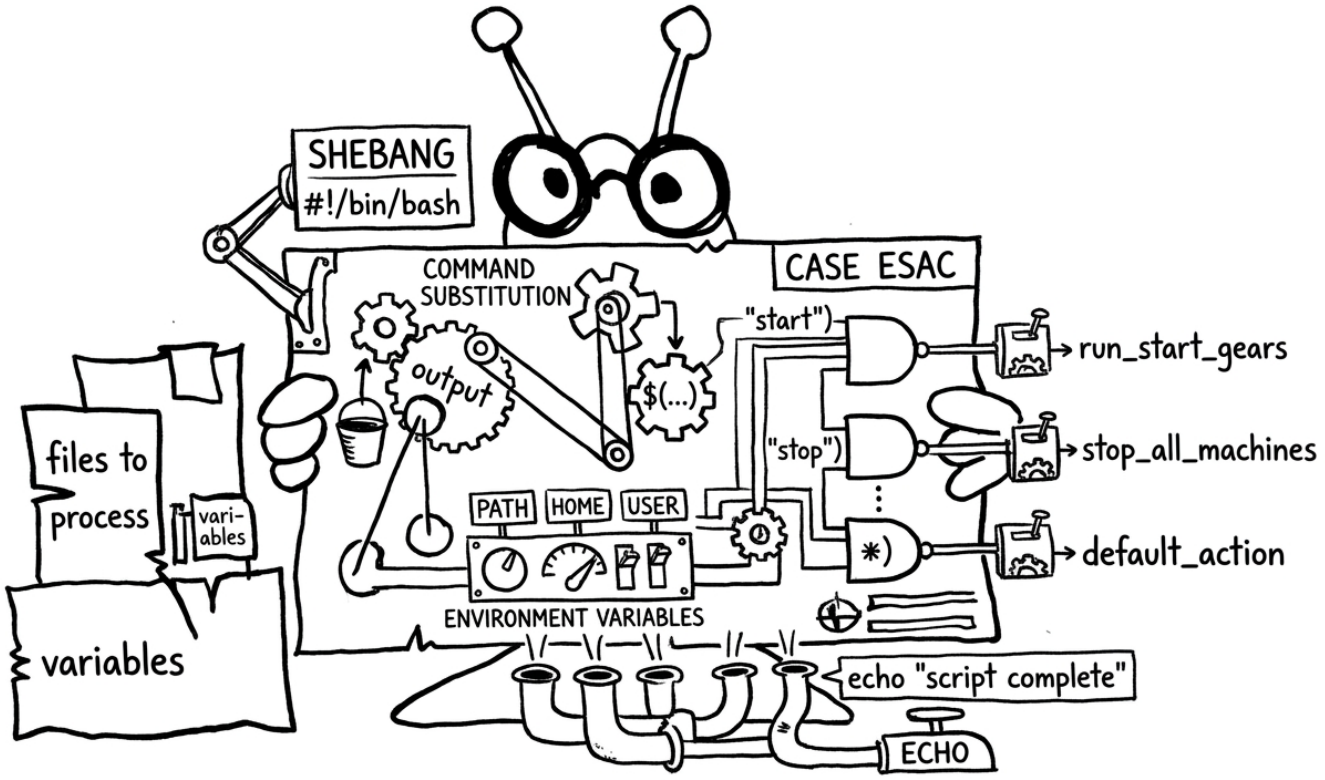
Chapitre 6: Programmation Shell



Objectifs du chapitre

1. Écrire un script shell avec shebang, permissions d'exécution et gestion du statut de sortie
2. Utiliser variables, substitution de commande, arithmétique et manipulation de chaînes
3. Implémenter des branchements avec if/elif/else, [[ ]] et case
4. Construire des **boucles** for, while, until sur des fichiers, tableaux ou valeurs calculées
5. Concevoir des **fonctions** avec variables locales et paramètres pour modulariser les scripts
6. Analyser les options de la ligne de commande avec getopts
7. Déboguer un script avec bash -x, set -e

Section 1 : Structure et exécution d'un script shell

Définition → shebang → permissions → méthodes d'exécution →
sous-shell vs source → statut de sortie → exemple hello.sh

Qu'est-ce qu'un script shell ?

Script shell : fichier texte contenant une suite de commandes que le shell exécute dans l'ordre

- Il enregistre la syntaxe que l'on ne veut pas retaper : il « mémorise » les commandes à votre place
- Un script peut utiliser à la fois les commandes internes du shell et les commandes externes (ls, mv, cp, rm, grep, find, cut, sed, echo)
- Tout programme installé peut être appelé comme commande dans un script
- Le shell ne « comprend » pas la commande : il lance le programme identifié par son nom
- Ajouter une commande dans un script = écrire sa ligne exacte dans le fichier

Le shebang et les commentaires

- **Shebang** : première ligne `#!/bin/bash` — deux caractères `#!` qui signalent un script, suivis du chemin absolu de l'interpréteur
- Le **noyau** lit cette ligne et lance `/bin/bash` pour interpréter le script
- **Autres noms** : hashbang, hashpling, pound-bang
- Après la première ligne, `#` commence un **commentaire** : le shell ignore le reste de la ligne

Les commentaires espacés améliorent la lisibilité ; un script doit rester lisible

Rendre le script exécutable : chmod +x

- Un script est un fichier ; il doit avoir le bit d'exécution pour être lancé directement
- `chmod u+x mon-script.sh` : ajoute l'exécution pour son propriétaire
- `chmod a+x mon-script.sh` : pour tous les utilisateurs
- Sans le bit **x**, `./mon-script.sh` échoue avec « **Permission denied** »

Alternative sans chmod : **bash mon-script.sh** (ne requiert pas le bit **x**)

Les six méthodes d'exécution

- `bash script.sh` : lance le script dans un sous-shell, sans exiger le bit **x**
- `./script.sh` : exige le bit **x**, crée un sous-shell, lance via le shebang
- `/chemin/script.sh` : identique à `./` mais par chemin absolu (utile hors du répertoire courant)
- `script.sh` : exige que le script soit dans un répertoire de **$PATH** (ex. /usr/local/bin)
- `sh script.sh` : quasi identique à `bash script.sh`
- `source script.sh` / `. script.sh` : n'exigent pas le bit **x** et ne créent pas de sous-shell

Sous-shell vs source : deux familles

- `bash script.sh` : lance le script dans un sous-shell, sans exiger le bit **x**
- `./script.sh` : exige le bit **x**, crée un sous-shell, lance via le shebang
- `/chemin/script.sh` : identique à `./` mais par chemin absolu (utile hors du répertoire courant)
- `script.sh` : exige que le script soit dans un répertoire de **$PATH** (ex. /usr/local/bin)
- `sh script.sh` : quasi identique à `bash script.sh`
- `source script.sh` / `. script.sh` : n'exigent pas le bit **x** et ne créent pas de sous-shell

Le statut de sortie et exit

Statut de sortie : valeur numérique (0–255) renvoyée par un programme à son parent

- Convention : **0** = succès, **valeur non nulle** = échec
- `$?` contient le statut de la dernière commande exécutée
- Dans un script: **exit** N termine le script et renvoie le code **N** ; exit seul renvoie le statut de la dernière commande
- Ce statut alimente les tests **if** et les listes `&&` / `||` — le fondement du contrôle de flux

Une commande qui échoue renvoie un code **spécifique**, pour réagir proprement

Exemple 1: premier script demo_v01.sh de bout en bout

- Cree un fichier nommee demo_v01.sh avec nano
- **Remplissez le avec** : `#!/bin/bash` puis `echo "Bonjour, $USER"` puis date puis `exit 0`
- **Rendre exécutable** : `chmod u+x demo_v01.sh`
- **Lancer** : `./demo_v01.sh` → **affiche le nom d'utilisateur et la date**
- **Vérifier le statut** : `echo $?` → 0
- **Tester le sourcing** : `source demo_v01.sh` → même résultat, dans le shell courant

Section 2 : Variables et traitement des données

Définition → guillemets → paramètres positionnels → substitution de commande
→ read → export → exemple mkuser.sh

Définir et développer une variable

- **Variable** : un emplacement nommé qui contient une valeur déterminée à l'exécution
- **Définition** : `nom="valeur"` — **sans dollar et sans espace autour du =**
- **Utilisation** : `$nom` ou `${nom}` — **avec dollar**

  Règle mnémotechnique : on agit sur la variable sans $, on agit avec la variable avec $

- `unset nom` **supprime la variable**
- **Nommage** : minuscules ou mixtes pour vos variables ; **les noms tout en majuscules** risquent d'**écraser** une variable d'environnement

Guillemets simples vs doubles

- **Guillemets doubles** "..." : le shell développe variables (**$var**) et substitutions de commande (**$(cmd)**) à l'intérieur
- **Guillemets simples** '...' : tout est **littéral**, aucun développement
- `echo "$count fichiers"` → développe la valeur de count
- `echo '$count fichiers'` → affiche littéralement $count fichiers
- Les guillemets protègent aussi les espaces et les caractères spéciaux

Les paramètres positionnels $1…$9

- **Paramètres positionnels** : les arguments passés au script sur la ligne de commande
- `$0` : **le nom du script** ; `$1` : **premier argument** ; `$2` : **second** ; **jusqu'à** `$9`
- **Au-delà de 9** : `${10}`, `${11}`, etc.
- Ils sont remplacés partout où ils apparaissent dans le script
- **Exemple** : `./mkuser.sh Ilias` → `$1` vaut `Ilias` dans tout le script
- Fondement de tout script paramétrable

La substitution de commande $(cmd)

- **Substitution de commande** : capturer la sortie (stdout) d'une commande dans une variable
- **Syntaxe moderne** : `variable=$(commande)`
- **Ancienne syntaxe** : `variable=`commande`` (backticks) — à éviter
- Le résultat remplace $(cmd) dans la ligne, comme si on l'avait tapé
- **Exemple** : `user_record=$(grep $USER /etc/passwd)` capture la ligne du fichier

Idéal pour stocker une liste, un résultat de calcul ou une information système

Lire une entrée : read

- **read** : lit une ligne depuis l'entrée standard et la range dans une variable
- read Username stocke la saisie dans Username, accessible ensuite via $Username
- Utile pour les scripts interactifs : demander une valeur à l'utilisateur
- Associer à echo -e "... \c" pour laisser le curseur sur la même ligne
- read -p "Nom : " nom : l'option moderne pour afficher l'invite sans echo

La variable non remplie (simple Entrée) reste vide

Variables d'environnement et export

- **Variable d'environnement** : une variable rendue visible aux sous-shells et aux programmes enfants
- `export VAR` (ou `export VAR=valeur`) la marque pour héritage
- Sans `export`, une variable reste locale au shell courant
- Script A lance Script B en sous-shell → B voit les variables exportées par A
- Convention : noms en majuscules pour l'environnement (`PATH`, `HOME`, `USER`)
- Exemple : `OSLINK=https://liaoux.github.io; export OSLINK`

Exemple 2 : demo_v02.sh — variables

- les variables — `nom="INF2132"`, `salutation="Bienvenue dans $nom"`
- `jour=$(date +%F)` : substitution de commande (date au format année-mois-jour)
- `echo "Premier argument : ${1:-aucun}"` : paramètre positionnel avec valeur par défaut
- `./demo_v02.sh Rapport` → affiche « Premier argument : Rapport »
- Sans argument, `${1:-aucun}` affiche « aucun »

Section 3 : Manipulation de chaînes

Longueur et sous-chaînes → suppression préfixe/suffixe → remplacement et défauts

Longueur et sous-chaînes ${#var} ${var:o:l}

- ${#var} : longueur de la valeur de var en nombre de caractères
- ${var:offset} : sous-chaîne depuis la position offset jusqu'à la fin
- ${var:offset:longueur} : sous-chaîne de longueur caractères à partir d'offset
- Les indices commencent à 0 ; un offset négatif compte depuis la fin
- Exemple : nom="Bonjour" → ${#nom} = 7, ${nom:0:3} = Bon

Aucune commande externe : tout est fait par bash, très rapide

Suppression de préfixe et de suffixe

- ${var#motif} : supprime le plus court préfixe (#) correspondant à motif
- ${var##motif} : supprime le plus long préfixe (##) correspondant
- ${var%motif} : supprime le plus court suffixe (%) correspondant
- ${var%%motif} : supprime le plus long suffixe (%%) correspondant
- **motif accepte les jokers** *, ?, [...]

Exemple de suppression de préfixe

Exemple: *chemin="/home/ilias/file.txt"*

- `${chemin##*/}` (Équivalent de la commande **basename**) —> file.txt
- **Explication** :
  - `${chemin ...}` : On manipule la variable ***chemin***.
  - `#` : Demande de **supprimer** un motif en partant du début de la chaîne (le prefixe).
  - `##` : Version « **gourmande** » (greedy), supprime la plus grande correspondance possible.
  - `*/` : Le motif à supprimer, c'est tout ce qui se termine par un slash / (donc tout le chemin du dossier : /home/ilias/).
- **En résumé** : On retire tout le chemin pour ne garder que ce qui se trouve après le dernier /: (file.txt)

Exemple de suppression de suffixe

Exemple: *chemin="/home/ilias/file.txt"*

- `${chemin%/*}` (Équivalent de la commande **dirname**) —> /home/ilias
- **Explication** :
  - `%` : Demande de supprimer un motif en partant de la fin de la chaîne (le suffixe).
  - `% (simple)` : Version non gourmande, supprime la plus petite correspondance.
  - `/*` : Le motif à supprimer en partant de la fin, c'est un slash suivi de n'importe quoi (/file.txt).
- En résumé : On retire le nom du fichier situé à la fin (après le dernier /) pour ne garder que le dossier parent.

Remplacement et valeurs par défaut

- ${var/ancien/nouveau} : remplace la première occurrence de ancien par nouveau
- ${var//ancien/nouveau} : remplace toutes les occurrences
- ${var:-défaut} : utilise défaut si var est vide ou non définie (sans l'affecter)
- ${var:=défaut} : idem, mais affecte défaut à var
- ${var:?message} : affiche message et termine si var est vide
- Usage type : config=${fichier:-/etc/monapp.conf}

Exemple : demo_v03.sh — manipulation de chaînes

Découper et transformer des chaînes

- **La chaîne à manipuler**: `fichier="photo.jpg"`:
  - `${fichier%.*}` → **photo** (base sans extension)
  - `${fichier##*.}` → **jpg** (extension)
- `chemin="/home/ilias/images/photo.jpg"`:
  - `${chemin##*/}` = **photo.jpg**
  - `${chemin%/*}` = **/home/ilias/images**

Section 4 : Les tableaux

Tableaux indexés → tableaux associatifs (introduction) → exemple demo_v04.sh

Tableaux indexés

- **Tableau indexé** : une variable contenant plusieurs **valeurs** repérées par un **entier**
- **Déclaration** : `arr=(a b c)` — les éléments séparés par des espaces
- **Accès** : `${arr[0]}` pour le premier élément (**indice** `0`)
- **Tous les éléments** : `"${arr[@]}"` — à toujours citer
- **Nombre d'éléments** : `${#arr[@]}`
- **Ajouter** : `arr+=(d)` ; **modifier** : `arr[1]=x`

Tableaux associatifs : introduction

- **Tableau associatif** : les éléments sont repérés par une clé textuelle, pas un entier
- **Déclaration obligatoire** : declare -A map
- **Affectation** : map[clé]=valeur
- **Lecture** : ${map[clé]}

```
#!/bin/bash

# Déclaration obligatoire du tableau associatif
declare -A capitales

# Affectation (clé textuelle -> valeur)
capitales[Maroc]="Rabat"
capitales[Espagne]="Madrid"
capitales[Japon]="Tokyo"
capitales[Canada]="Ottawa"

# Lecture des informations
echo "La capitale du Maroc est : ${capitales[Maroc]}"
echo "La capitale du Japon est : ${capitales[Japon]}"

# Modification d'une valeur existante
capitales[Maroc]="Casablanca (exemple modifié)"
echo "Nouvelle valeur pour le Maroc: ${capitales[Maroc]}"
```

Principales options de declare

declare: Une commande Bash permet de déclarer des variables et de leur attribuer des propriétés ou des comportements spécifiques.

Option	Signification	Exemple
-A (Tableau associatif)	Déclare un tableau dont les index sont des clés textuelles et non des nombres.	declare **-A** config=([hote]="localhost" [port]=8080)
-a (Tableau indexé)	Déclare explicitement un tableau classique (à indices numériques). (utile pour forcer le type).	declare **-a** fruits=("pomme" "banane" "cerise")
-i (Integer)	Traite la variable comme un nombre entier. Toute affectation subira une évaluation arithmétique automatique.	declare **-i** resultat resultat=5+5 # resultat vaudra 10
-r (Read-only)	Rend la variable immuable. Sa valeur ne pourra plus être modifiée ni la variable supprimée par la suite.	declare **-A** config=([hote]="localhost" [port]=8080)

Exemple : demo_v04.sh — tableaux indexés

- `fichiers=(*.txt)` : le globbing remplit le tableau avec les fichiers `.txt`
- `${#fichiers[@]}` → nombre d'éléments (3 fichiers d'exemple)
- `${fichiers[0]:-aucun}` → premier élément, avec valeur par défaut
- Le tableau capture une liste à traiter sans reparcourir le répertoire

Quiz 1

Q1. La ligne `#!/bin/bash` en tête d'un script est…

A. *le shebang* · ***B.*** *un commentaire* · ***C.*** *une variable* · ***D.*** *une commande*

Q2. `chmod u+x script.sh` …

A. *rend le script lisible* · ***B.*** *rend le script exécutable pour son propriétaire* · ***C.*** *supprime le script*

Q3. `./script.sh` se distingue de `source script.sh` car…

A. *il exige le bit x* · ***B.*** *il crée un sous-shell* · ***C.*** *ses variables ne persistent pas* · ***D.*** *A, B et C*

Q4. `${chemin##*/}` pour `chemin="/etc/ssh/sshd_config"` vaut…

A. */etc/ssh* · ***B.*** *sshd_config* · ***C.*** *config* · ***D.*** */etc/ssh/sshd_config*

Section 5 : Structures conditionnelles

if/then → test et [ ] → comparaisons → tests de fichiers → elif et [[ ]] → case → exemple demo_v05.sh

if/then/fi : tester le succès d'une commande

Syntaxe : if commande ; then ... fi

- Le if teste le **statut de sortie** de commande, pas une valeur booléenne
- **Succès** (code 0) → le bloc then s'exécute
- **Échec** (code non nul) → on passe au bloc else s'il existe
- fi clôt la structure (c'est if à l'envers)
- **Exemple** : **if** date ; **then** echo "Cela fonctionne" ; **fi**

La commande test et les crochets [ ]

test : commande qui évalue une condition et renvoie **0 (vrai)** ou non **nul (faux)**

- test "$var" : **vrai** si $var n'est pas vide
- **[condition]** : notation équivalente à test condition
- Piège : espaces obligatoires après [et avant]
- **Trois familles de tests** : numériques, chaînes, fichiers
- **test** combine avec && (**et**) et || (**ou**)

Comparaisons numériques et de chaînes

- **Numériques** : -eq, -ne, -lt, -le, -gt, -ge (equal, not equal, less than…)
- **Pour les entiers**, déclarer declare **-i** var évite les comparaisons inattendues
- **Chaînes** : = (égal), != (différent), -z (vide), -n (non vide)
- **Exemple** : ["$count" -gt 5], ["$nom" = "root"]
- Ne pas confondre = (chaînes) et -eq (nombres)
- Toujours citer les variables pour éviter « unary operator expected »

Tests de fichiers -f -d -r...

- `-e` : le fichier existe ; `-f` : fichier régulier ; `-d` : répertoire
- `-r`, `-w`, `-x` : lisible, inscriptible, exécutable
- `-s` : fichier non vide ; `-L` : lien symbolique
- **Exemple** : `if [ -f "$fichier" ] ; then echo "existe" ; fi`
- Indispensable pour valider les arguments d'un script avant de les traiter
- Les tests de fichiers se combinent avec `&&` et `||`

if/elif/else, [[ ]], && et | |

elif : branche conditionnelle supplémentaire entre if et else

- **[[ ]]** : version bash étendue du test — **motifs**, =~ (regex), pas de découpage de mots
- [["$mot" =~ ^[0-9]+$]] : teste une chaîne par expression régulière
- [a] **&&** [b] : les deux conditions doivent être vraies
- [a] **| |** [b] : l'une des deux suffit

case/esac : aiguillage multiple

- **case mot in motif) commandes ;; esac : compare mot à des motifs successifs**
- Les motifs acceptent les **jokers** *, ?, [...]
- Chaque branche se termine par ;; la structure se clôt par esac (case à l'envers)
- *) : le motif attrape-tout pour les cas **par défaut**
- Plus lisible que des elif en cascade dès qu'il y a plus de **deux valeurs**
- **Remarque** : Dans case on utilise le symbole | (séparateur de motifs, équivalent à "**OU**") et non || pour lister plusieurs choix dans un même motif (ex: start|restart)

Exemple: if/then/elif/fi vs case/esac

```bash
#!/bin/bash

read -p "Entrez le nom d'un fichier : " fichier

if [[ "$fichier" == *.txt ]]; then
    echo "C'est un fichier texte."
elif [[ "$fichier" == *.sh ]]; then
    echo "C'est un script Bash."
elif [[ "$fichier" == *.jpg ]] || [[ "$fichier" == *.png ]]; then
    echo "C'est une image."
else
    echo "Format de fichier inconnu."
fi
```

```bash
#!/bin/bash

read -p "Entrez le nom d'un fichier : " fichier

case "$fichier" in
    *.txt)
        echo "C'est un fichier texte." ;;
    *.sh)
        echo "C'est un script Bash." ;;
    *.jpg | *.png)
        echo "C'est une image." ;;
    *)
        echo "Format de fichier inconnu." ;;
esac
```

Exemple : demo_v05.sh — conditions

- demo_v05.sh: réagir selon le type de la cible passée en argument
- cible="${1:-.}" : la cible est l'argument **1**, sinon le répertoire courant **.**
- if [-d "$cible"] / elif [-f "$cible"] / else : teste répertoire, fichier, sinon introuvable
- case "$cible" in *.txt) … ;; *) … ;; esac : reconnaît un fichier texte
- ./demo_v05.sh → « . est un répertoire » ; ./demo_v05.sh photo.txt → « photo.txt est un fichier »

Section 6 : Les boucles

while → until → for sur liste → for en style C → select → exemple demo_v06.sh → Quiz 2

while : répéter tant que vrai

- **Syntaxe** : **while** [condition] ; **do** commandes ; **done**
- Le **corps s'exécute** tant que la condition est **vraie**
- Idéal pour répéter tant qu'une condition persiste (utilisateur n'a pas dit « Exit »)
- while ["$Username" != "Exit"] : continue tant que l'utilisateur ne tape pas Exit
- **Se combine avec read pour lire un fichier ligne par ligne**
- **Toujours citer les variables dans la condition**

until et le piège des guillemets

- **Syntaxe** : **until** [condition] ; **do** commandes ; **done**
- Le **corps s'exécute** jusqu'à ce que la condition **devienne vraie**
- Logique **inverse** de **while** : on répète tant que la condition est **fausse**
- **while** ["$u" != "Exit"] ≡ **until** ["$u" = "Exit"]
- Piège : [$u != "Exit"] avec u vide → « unary operator expected »
- Remède : citer "$u" ou utiliser [[ ]]

Exemple: while vs until

```
#!/bin/bash
Username=""
while [ "$Username" != "Exit" ]; do
    read -p "Entrez votre nom d'utilisateur (ou 'Exit' pour quitter) : " Username
    echo "Vous avez saisi : $Username"
done
echo "Boucle terminée. Au revoir !"
```

While Demande un nom d'utilisateur jusqu'à ce qu'il tape "Exit". La boucle tourne tant que $Username est différent de "Exit".

```
#!/bin/bash
Username=""
until [ "$Username" = "Exit" ]; do
    read -p "Entrez votre nom d'utilisateur (ou 'Exit' pour quitter) : " Username
    echo "Vous avez saisi : $Username"
done
echo "Boucle terminée. Au revoir !"
```

Until s'exécute tant que la condition est fausse. La boucle tourne jusqu'à ce que $Username soit égal à "Exit".

for sur une liste

- **Syntaxe** : for var in liste ; do commandes ; done
- À chaque itération, **var** prend la valeur de l'élément suivant de **la liste**
- Pas de condition : la boucle parcourt simplement tous les éléments
- La liste peut être littérale, un globbing (*.txt), ou le résultat d'une commande
- for f in *.txt ; do wc -l "$f" ; done : traite chaque fichier
- C'est la boucle privilégiée pour les fichiers et les tableaux

Le for style C (syntaxe arithmétique)

- **Syntaxe** : for ((initialisation ; condition ; incrémentation)) ; do commandes ; done
- **Exemple classique** : for ((i=1; i<=5; i++)) ; do echo "Tour numéro $i" ; done
- Permet de répéter un bloc un nombre précis de fois en pilotant un compteur numérique.
- **Initialisation** : exécutée une seule fois au début (ex. i=1).
- **Condition** : évaluée avant chaque itération ; si elle est fausse, la boucle s'arrête (ex. i <= 5).
- Incrémentation : exécutée à la fin de chaque tour (ex. i++).
- **Attention** : Utilise les doubles parenthèses (( )) pour l'évaluation arithmétique (pas besoin de mettre de signe $ devant les variables à l'intérieur).

Exemples: Le for style C

- **Syntaxe** : **for** ((initialisation ; condition ; incrémentation)) ; **do** commandes ; **done**
- **Exemple classique** : **for** ((i=1; i<=5; i++)) ; **do** echo "Tour numéro $i" ; **done**
- Permet de répéter un bloc un nombre précis de fois en pilotant un compteur numérique.
- **Initialisation** : exécutée une seule fois au début (ex. i=1).
- **Condition** : évaluée avant chaque itération ; si elle est fausse, la boucle s'arrête (ex. i <= 5).
- Incrémentation : exécutée à la fin de chaque tour (ex. i++).
- **Attention** : Utilise les doubles parenthèses (( )) pour l'évaluation arithmétique (pas besoin de mettre de signe $ devant les variables à l'intérieur).

Exemple: list-base vs arithmetic-base for loop

```bash
#!/bin/bash

fruits=("Pomme" "Banane" "Cerise" "Date")

for fruit in "${fruits[@]}"; do
    echo "First for loop"
    echo "I like eating: $fruit"
done
echo "-----------------------------------"
# --- La deuxième boucle (style C) ---
for (( i=0; i<${#fruits[@]}; i++ )); do
    echo "Second for loop"
    echo "Fruit à l'indice $i : ${fruits[$i]}"
done
```

Simple for (sur un tableau) : Toujours utiliser for item in "${tableau[@]}"; do ... done.

for style C : Parfait pour cibler des indices précis, faire des pas (ex: i+=2), ou parcourir un tableau de la fin vers le début.

select : un menu interactif

- **select** var **in** liste ; **do** … ; **done** : affiche un menu numéroté à partir de la liste
- L'utilisateur **tape** un numéro, et **var** reçoit l'élément correspondant
- Se combine avec **case** pour réagir au **choix**
- PS3 : variable d'invite du menu (PS3="Votre choix : ")
- **Exemple** : select action in "**Démarrer**" "**Arrêter**" "**Quitter**"
- Idéal pour les scripts interactifs simples, sans saisie de texte libre

Exemple: select et case

```
#!/bin/bash
PS3="Votre choix : "
echo "=== GESTIONNAIRE DE SERVICE ==="
# Affichage automatique d'un menu numéroté à partir de la liste
select action in "Démarrer" "Arrêter"; do
  case "$action" in
    "Démarrer")
      echo "Le service a été démarré avec succès.";;
    "Arrêter")
      echo "Le service a été arrêté.";;
      break;;
    *)
      # Gère les choix invalides (ex: si l'utilisateur tape un chiffre hors liste ou du texte)
      echo "Choix invalide ($REPLY). Veuillez entrer un numéro entre 1 et 2.";;
  esac
done
```

1. `PS3` : Personnalise le texte de l'invite de commande du menu.

2. `Association Numéro/Valeur` : L'utilisateur tape un chiffre, mais la variable reçoit directement le texte correspondant.

3. **Variable** `$REPLY` : Stocke la saisie brute en cas de choix invalide, idéal pour la gestion des erreurs.

4. **Nécessité du** `break` : Indispensable pour quitter la boucle, car select tourne indéfiniment par défaut.

Exemple : demo_v06.sh — boucles

- **Objectif**: répéter un traitement sur plusieurs fichiers
- `for f in *.txt ; do echo " - $f" ; done` : parcourt chaque fichier .txt
- `n=1` puis `while [ "$n" -le 3 ] ; do echo "Compteur : $n" ; n=$((n + 1)) ; done`
- Le `for` itère sur la liste ; le `while` répète tant que la condition est vraie
- `$((n + 1))` : expansion arithmétique pour incrémenter

Quiz 2

Q1. Pour comparer deux entiers, on emploie…

A. *=* · ***B***. *-eq* · ***C***. *==* · ***D***. *-z*

Q2. La boucle qui parcourt les éléments d'une liste est…

A. *while* · ***B***. *until* · ***C***. *for* · ***D***. *case*

Q3. Une cascade de elif sur une même variable se remplace avantageusement par…

A. *for* · ***B***. *case* · ***C***. *read* · ***D***. *export*

Q4. while ["$u" != "Exit"] s'écrit de façon équivalente…

A. *until ["$u" = "Exit"]* · ***B***. *for "$u"* · ***C***. *case "$u"* · ***D***. *if "$u"*

Section 7 : Les fonctions

Définir et appeler → paramètres et local → sourcer un fichier → exemple demo_v07.sh

Définir et appeler une fonction

- **Fonction** : bloc nommé qui effectue une sous-tâche et s'appelle comme une commande
- **Définition** : nom() { commandes ; } (le mot-clé function est optionnel)
- **Appel** : écrire simplement nom dans le script
- Une fonction n'est exécutée que lorsqu'elle est appelée, pas à sa définition
- Regrouper le code répété (messages, nettoyage) en fonctions rend le script modulaire
- **Exemple : erreur() { echo "Une erreur est survenue" ; exit 1 ; }**

Paramètres de fonction et variables locales

- **Dans une fonction**, `$1`, `$2`… désignent les **arguments** de la fonction, ~~pas ceux du script~~
- `$#` donne le nombre d'arguments de la fonction ; `$@` tous les arguments
- `local var` limite la portée d'une variable à la fonction (sinon elle est globale)
- `return N` termine la fonction et fixe son statut de sortie
- **Piège** : une variable **non déclarée local** « **fuit** » **hors de la fonction**
- **Exemple** : **usage() { echo "Usage : $0 [-v] fichier" ; return 1 ; }**

Sourcer un fichier de fonctions

- Regrouper des fonctions réutilisables dans un fichier dédié, ex. lib.sh
- source lib.sh (ou . lib.sh) charge les fonctions dans le script courant
- Évite de recopier le même code dans chaque script
- **Exemple** : source ./fonctions.sh puis appeler **usage()** ou **is_root()**

Constitue une mini-bibliothèque de fonctions personnelles

Exemple : demo_v07.sh — fonctions

- Organiser le code en fonctions réutilisables
- usage() { echo "Usage : $0 [cible]" ; exit 1 ; } : affiche l'aide et sort
- log() { local msg="$1" ; echo "[INFO] $msg" ; } : journalise avec une variable locale
- [$# -gt 1] && usage : appelle **usage()** si plus d'un argument est fourni
- **Appels** : log "Démarrage du script" … log "Fin du script" encadrent le corps
- local limite **msg** à la fonction ; $0 reste le nom du script

Section 8 : Paramètres positionnels, getopts et debogage

Paramètres spéciaux → $@ vs $* → shift → getopts → bash -x

Paramètres spéciaux : $0, $#, $@, $*

- $0 : nom du script (tel qu'il a été invoqué)
- $1…$9, ${10} : arguments positionnels
- $# : nombre total d'arguments
- $@ : tous les arguments, chacun comme un mot distinct
- $* : tous les arguments, concaténés en un seul mot
- Ces variables alimentent la validation et l'itération sur les arguments

"$@" vs "$*"

- "$@" : chaque argument reste un mot séparé — c'est la forme à utiliser presque toujours
- "$*" : tous les arguments fusionnés en une seule chaîne
- Sans guillemets, $@ et $* se comportent de façon identique (découpage en mots)
- for a in "$@" ; do … done : itère correctement sur chaque argument
- Avec "$*", une itération ne ferait qu'un seul tour avec tout concaténé
- Règle : préférez "$@" pour transmettre ou parcourir les arguments

"$@" vs "$*"

- "$@" : chaque argument reste un mot séparé — c'est la forme à utiliser presque toujours
- "$*" : tous les arguments fusionnés en une seule chaîne
- Sans guillemets, $@ et $* se comportent de façon identique (découpage en mots)
- for a in "$@" ; do … done : itère correctement sur chaque argument
- Avec "$*", une itération ne ferait qu'un seul tour avec tout concaténé
- Règle : préférez "$@" pour transmettre ou parcourir les arguments

`shift` : décaler les paramètres

- **shift** : supprime `$1` et décale les suivants — `$2` devient `$1`, `$3` devient `$2`, etc.

  Permet de traiter les arguments un à un sans connaître leur nombre à l'avance

- Après le dernier paramètre, `$1` devient **vide** — on le teste pour arrêter
- `shift N` : décale de **N** positions d'un coup

Exemple: shift en action

```
#!/bin/bash
echo "=== Début du script ==="
echo "Nombre total d'arguments reçus : $# "
echo "-------------------------------------"
# Boucle tant que $1 n'est pas vide
while [ "$1" != "" ]; do
   echo "1. \$1 actuel contient : '$1'"
   echo "   (Les arguments suivants étaient : \$2='$2', \$3='$3'...)"
    # Simulation d'un traitement sur l'argument courant ($1)
   echo "   -> Traitement en cours de : $1..."
    # On décale tout d'un cran vers la gauche
   shift
   echo "   -> Après 'shift', le nouveau \$1 est : '$1'"
done

echo "=== Fin : Tous les arguments ont été consommés ==="
```

Sans shift, si un utilisateur passe 2, 5 ou 50 arguments à votre script, vous seriez obligé de faire des vérifications complexes sur **$1, $2, $3... jusqu'à $50.**

Grâce à shift, vous traitez toujours la même variable (**$1**), et la liste des arguments se "consomme" d'elle-même au fur et à mesure !

getopts : analyser les options

- getopts : analyse les options de la ligne de commande selon une chaîne de spécification
- getopts "vo:f:" opt : v sans argument, o: et f: avec argument
- $opt reçoit l'option courante ; $OPTARG l'argument de l'option
- $OPTIND : index de l'argument suivant à traiter
- Se combine avec un **case** pour traiter chaque option
- shift $((OPTIND-1)) : laisse les arguments restants dans $1, $2…

Exemple: getopts en action: Voir demo_getopts.sh

La fonction usage() :Centralise l'affichage de l'aide en cas de mauvaise manipulation ou si l'utilisateur demande -h. Elle utilise **$0** pour afficher le nom du script et se termine par **exit 1** pour signaler un arrêt suite à une erreur d'utilisation.

La boucle while getopts "vhf:" opt :Analyse les options transmises à la ligne de commande.

- v et h s'utilisent seuls (drapeaux).
- f: est suivi d'un : ce qui oblige l'utilisateur à lui fournir une valeur (stockée automatiquement dans **$OPTARG**).

L'aiguillage case "$opt" :Permet d'exécuter le code correspondant à chaque option trouvée. Le cas **\?)** capture les erreurs (comme une option -x inexistante ou l'oubli de l'argument de -f) pour appeler automatiquement la fonction usage.

Le décalage shift $((OPTIND - 1)) :Utilise la variable système **$OPTIND** (qui indique l'index du prochain élément à lire) pour décaler et nettoyer toutes les options déjà traitées. Seuls les arguments positionnels bruts restants (s'il y en a) demeurent dans **$@**.

Debogage avec le mode trace : bash -x vs set -x

xtrace : mode de trace qui affiche chaque commande après développement, préfixée de +

Lancement externe (bash -x script.sh) :

- Exécute le script en affichant chaque commande précédée d'un signe + avant de l'exécuter.
- Idéal pour tester sans modifier le fichier.

Insertion interne (set -x / set +x) :

- set -x : Active le mode trace à partir de cette ligne dans le script.
- set +x : Désactive le mode trace plus loin dans le script.
- Utile pour cibler uniquement une portion suspecte du code.

Exemple : debogage avec bash -x

```
!\bash\bin\
if [ $# == 0 ]; then
        echo "Usage: $0 <extension>"
        exit 0
fi
extension=$2
echo "Recherche des fichiers *.$extension ..."
for fichier in *.$extension; do
   # Teste si le fichier trouvé est un répertoire (dossier)
        if [ fichier == -d ]; then
                echo "Trouvé (dossier): &fichier"
        fi
done
```

Lancer: bash -x cherche_bug.sh txt

Quiz: Programmation Shell



https://forms.cloud.microsoft/r/7R3Lcrjnyi